

Collections in Kotlin

Link: <https://developer.android.com/codelabs/basic-android-kotlin-training-collections>

1. Before you begin
2. Learn about collections
3. Working with collections
4. Learn about lambdas and higher-order functions
5. Make word lists
6. Summary
7. Learn more

About this codelab

subject Last updated Oct 20, 2023

account_circle Written by Google Developers Training team

1. Before you begin

Caution: This codelab is out of date and no longer maintained. Instead, please refer to the [Android Basics with Compose](#) course for the latest recommended practices.

In this codelab you will learn more about collections, and about lambdas and higher-order functions in Kotlin.

Prerequisites

- A basic understanding of Kotlin concepts as presented in the prior codelabs.
- Familiar with using the [Kotlin Playground](#) for creating and editing Kotlin programs.

What you'll learn

- How to work with collections including sets and maps
- The basics of lambdas
- The basics of higher-order functions

What you need

- A computer with an internet connection to access the [Kotlin Playground](#)

2. Learn about collections

A [collection](#) is a group of related items, like a list of words, or a set of employee records. The collection can have the items ordered or unordered, and the items can be unique or not. You've already learned about one type of collection, lists. Lists have an order to the items, but the items don't have to be unique.

As with lists, Kotlin distinguishes between mutable and immutable collections. Kotlin provides numerous functions for adding or deleting items, viewing, and manipulating collections.

Create a list

In this task you'll review creating a list of numbers and sort them.

1. Open the [Kotlin Playground](#).
2. Replace any code with this code:

```
fun main() {  
    val numbers = listOf(0, 3, 8, 4, 0, 5, 5, 8, 9, 2)  
    println("list:    ${numbers}")  
}
```

3. Run the program by tapping the green arrow, and look at the results that appear:

```
list:    [0, 3, 8, 4, 0, 5, 5, 8, 9, 2]
```

4. The list contains 10 numbers from 0 to 9. Some of the numbers appear more than once while some don't appear at all.
5. The order of the items in the list matters: the first item is 0, the second item is 3, and so on. The items will stay in that order unless you change them.
6. Recall from earlier codelabs that lists have many built-in functions, like `sorted()`, which returns a copy of the list sorted in ascending order. After the `println()`, add a line to your program to print a sorted copy of the list:

```
println("sorted: ${numbers.sorted()}")
```

7. Run your program again and look at the results:

```
list:    [0, 3, 8, 4, 0, 5, 5, 8, 9, 2]  
sorted:  [0, 0, 2, 3, 4, 5, 5, 8, 8, 9]
```

With the numbers sorted, it's easier to see how many times each number appears in your list, or if it doesn't appear at all.

Learn about sets

Another type of collection in Kotlin is a [set](#). It's a group of related items, but unlike a list, there can't be any duplicates, and the order doesn't matter. An item can be in the set or not, but if it's in the set, there is only one copy of it. This is similar to the mathematical concept of a set. For example, there is a set of books that you've read. Reading a book multiple times doesn't change the fact it is in the set of books that you've read.

1. Add these lines to your program to convert the list to a set:

```
val setOfNumbers = numbers.toSet()
println("set:      ${setOfNumbers}")
```

2. Run your program and look at the results:

```
list:    [0, 3, 8, 4, 0, 5, 5, 8, 9, 2]
sorted:  [0, 0, 2, 3, 4, 5, 5, 8, 8, 9]
set:     [0, 3, 8, 4, 5, 9, 2]
```

The result has all the numbers in the original list, but each only appears once. Note that they are in the same order as in the original list, but that order isn't significant for a set.

3. Define a mutable set and an immutable set, and initialize them with the same set of numbers but in a different order by adding these lines:

```
val set1 = setOf(1,2,3)
val set2 = mutableSetOf(3,2,1)
```

4. Add a line to print whether they are equal:

```
println("$set1 == $set2: ${set1 == set2}")
```

5. Run your program and look at the new results:

```
[1, 2, 3] == [3, 2, 1]: true
```

Even though one is mutable and one isn't, and they have the items in a different order, they're considered equal because they contain exactly the same set of items.

One of the main operations you might perform on a set is checking if a particular item is in the set or not with the [contains\(\)](#) function. You've seen `contains()` before, but used it on a list.

6. Add this line to your program to print if 7 is in the set:

```
println("contains 7: ${setOfNumbers.contains(7)}")
```

7. Run your program and look at the additional results:

```
contains 7: false
```

You can try testing it with a value that is in the set, too.

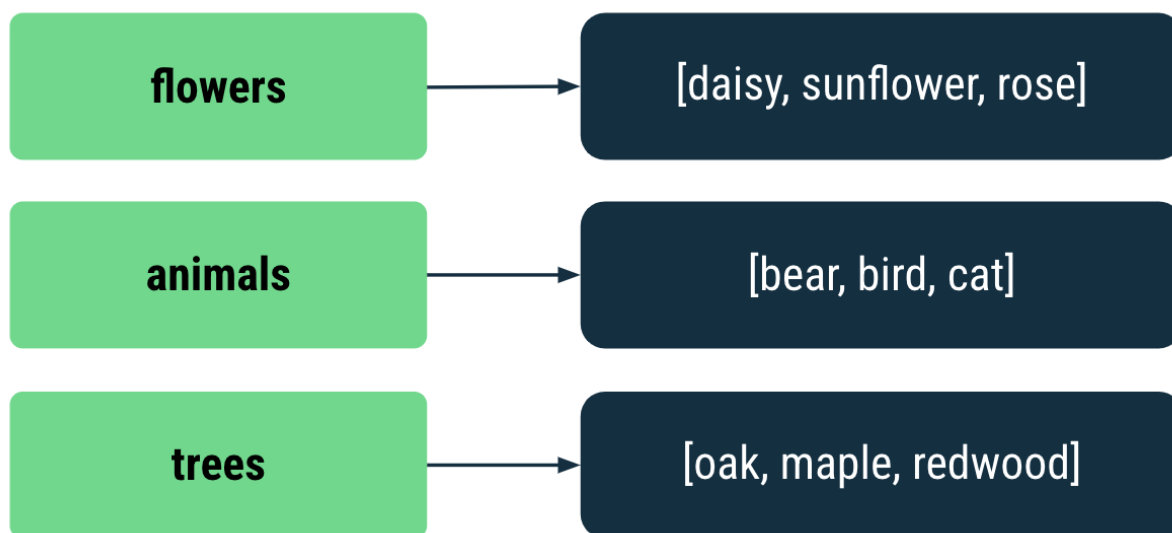
All of the code above:

```
fun main() {
    val numbers = listOf(0, 3, 8, 4, 0, 5, 5, 8, 9, 2)
    println("list:    ${numbers}")
    println("sorted:  ${numbers.sorted()}")
    val setOfNumbers = numbers.toSet()
    println("set:      ${setOfNumbers}")
    val set1 = setOf(1,2,3)
    val set2 = mutableSetOf(3,2,1)
    println("$set1 == $set2: ${set1 == set2}")
    println("contains 7: ${setOfNumbers.contains(7)}")
}
```

As with mathematical sets, in Kotlin you can also perform operations like the intersection ($\cap$) or the union ($\cup$) of two sets, using [intersect\(\)](#) or [union\(\)](#).

Learn about maps

The last type of collection you'll learn about in this codelab is a [map](#) or *dictionary*. A map is a set of *key-value pairs*, designed to make it easy to look up a value given a particular key. Keys are unique, and each key maps to exactly one value, but the values can have duplicates. Values in a map can be strings, numbers, or objects—even another collection like a list or a set.



A map is useful when you have pairs of data, and you can identify each pair based on its key. The key "maps to" the corresponding value.

1. In the Kotlin playground, replace all the code with this code that creates a mutable map to store people's names and their ages:

```
fun main() {
    val peopleAges = mutableMapOf<String, Int>(
        "Fred" to 30,
        "Ann" to 23
    )
    println(peopleAges)
}
```

This creates a mutable map of a `String` (key) to an `Int` (value), initializes the map with two entries, and prints the items.

2. Run your program and look at the results:

```
{Fred=30, Ann=23}
```

3. To add more entries to the map, you can use the [put\(\)](#) function, passing in the key and the value:

```
peopleAges.put("Barbara", 42)
```

4. You can also use a shorthand notation to add entries:

```
peopleAges["Joe"] = 51
```

Here is all the of the code above:

```
fun main() {
    val peopleAges = mutableMapOf<String, Int>(
        "Fred" to 30,
        "Ann" to 23
    )
    peopleAges.put("Barbara", 42)
    peopleAges["Joe"] = 51
    println(peopleAges)
}
```

5. Run your program, and look at the results:

```
{Fred=30, Ann=23, Barbara=42, Joe=51}
```

As noted above, the keys (names) are unique, but the values (ages) can have duplicates. What do you think happens if you try to add an item using one of the same keys?

6. Before the `println()`, add this line of code:

```
peopleAges["Fred"] = 31
```

7. Run your program, and look at the results:

```
{Fred=31, Ann=23, Barbara=42, Joe=51}
```

The key "Fred" doesn't get added again, but the value it maps to is updated to 31.

As you can see, maps are useful as a quick way to map keys to values in your code!

3. Working with collections

Although they have different qualities, different types of collections have a lot of behavior in common. If they're mutable, you can add or remove items. You can enumerate all the items, find a particular item, or sometimes convert one type of collection to another. You did this earlier where you converted a `List` to a `Set` with [toSet\(\)](#). Here are some helpful functions for working with collections.

forEach

Suppose you wanted to print the items in `peopleAges`, and include the person's name and age. For example, "Fred is 31, Ann is 23, ..." and so on. You learned about `for` loops in an earlier codelab, so you could write a loop with `for (people in peopleAges) { ... }`.

However, enumerating all the objects in a collection is a common operation, so Kotlin provides [forEach\(\)](#), which goes through all the items for you and performs an operation on each one.

1. In the playground, add this code after the `println()`:

```
peopleAges.forEach { print("${it.key} is ${it.value}, ") }
```

It's similar to the `for` loop, but a little more compact. Instead of you specifying a variable for the current item, the `forEach` uses the special identifier `it`.

Note that you didn't need to add parentheses when you called the `forEach()` method, just pass the code in curly braces `{}`.

2. Run your program and look at the additional results:

```
Fred is 31, Ann is 23, Barbara is 42, Joe is 51,
```

That's very close to what you want, but there's an extra comma on the end.

Converting a collection into a string is a common operation, and that extra separator at the end is a common problem, too. You'll learn how to deal with that in the steps ahead.

map

The [map\(\)](#) function (which shouldn't be confused with a map or dictionary collection above) applies a transformation to each item in a collection.

1. In your program, replace the `forEach` statement with this line:

```
println(peopleAges.map { "${it.key} is ${it.value}" }.joinToString(", ") )
```

2. Run your program and look at the additional results:

```
Fred is 31, Ann is 23, Barbara is 42, Joe is 51
```

It has the correct output, and no extra comma! There's a lot going on in one line, so take a closer look at it.

- `peopleAges.map` applies a transformation to each item in `peopleAges` and creates a new collection of the transformed items
- The part in the curly braces `{}` defines the transformation to apply to each item. The transformation takes a key value pair and transforms it into a string, for example `<Fred, 31>` turns into `Fred is 31`.
- `joinToString(", ")` adds each item in the transformed collection to a string, separated by `,` and it knows not to add it to the last item
- all this is chained together with `.` (dot operator), like you've done with function calls and property accesses in earlier codelabs

filter

Another common operation with collections is to find the items that match a particular condition. The [`filter\(\)`](#) function returns the items in a collection that match, based on an expression.

1. After the `println()`, add these lines:

```
val filteredNames = peopleAges.filter { it.key.length < 4 }  
println(filteredNames)
```

Again note that the call to `filter` doesn't need parentheses, and `it` refers to the current item in the list.

2. Run your program and look at the additional results:

```
{Ann=23, Joe=51}
```

In this case, the expression gets the length of the key (a `String`) and checks if it is less than 4. Any items that match, that is, have a name with fewer than 4 characters, are added to the new collection.

The type returned when you applied the filter to a map is a new map (`LinkedHashMap`). You could do additional processing on the map, or convert it to another type of collection like a list.

4. Learn about lambdas and higher-order functions

Lambdas

Let's revisit this earlier example:

```
peopleAges.forEach { print("${it.key} is ${it.value}") }
```

There's a variable (`peopleAges`) with a function (`forEach`) being called on it. Instead of parentheses following the function name with the parameters, you see some code in curly braces `{ }` following the function name. The same pattern appears in the code that uses `map` and `filter` functions from the previous step. The `forEach` function gets called on the `peopleAges` variable and uses the code in the curly braces.

It's like you wrote a small function in the curly braces, but there's no function name. This idea—a function with no name that can immediately be used as an expression—is a really useful concept called a *lambda expression*, or just *lambda*, for short.

This leads to an important topic of how you can interact with functions in a powerful way with Kotlin. You can store functions in variables and classes, pass functions as arguments, and even return functions. You can treat them like you would variables of other types like `Int` or `String`.

Function types

To enable this type of behavior, Kotlin has something called *function types*, where you can define a specific type of function based on its input parameters and return value. It appears in the following format:

Example Function Type: `(Int) -> Int`

A function with the above function type must take in a parameter of type `Int` and return a value of type `Int`. In function type notation, the parameters are listed in parentheses (separated by commas if there are multiple parameters). Then there is an arrow `->` which is followed by the return type.

What type of function would meet this criteria? You could have a lambda expression that triples the value of an integer input, as seen below. For the syntax of a lambda expression, the parameters come first (highlighted in the red box), followed by the function arrow, and followed by the function body (highlighted in the purple box). The last expression in the lambda is the return value.

```
{ a: Int -> a * 3 }
```

You could even store a lambda into a variable, as shown in the below diagram. The syntax is similar to how you declare a variable of a basic data type like an `Int`. Observe the variable name (yellow box), variable type (blue box), and variable value (green box). The `triple` variable stores a function. Its type is a function type `(Int) -> Int`, and the value is a lambda expression `{ a: Int -> a * 3 }`.

1. Try this code in the playground. Define and call the `triple` function by passing it a number like 5.

```
val number: Int = 5
```



```
val triple: (Int) -> Int = { a: Int -> a * 3 }
```

Function TypeLambda

```
fun main() {  
    val triple: (Int) -> Int = { a: Int -> a * 3 }  
    println(triple(5))  
}
```

2. The resulting output should be:

15

Note: It's common to have a lambda that has a single parameter, so Kotlin offers a shorthand. Kotlin implicitly uses the special identifier `it` for the parameter of a lambda with a single parameter.

3. Within the curly braces, you can omit explicitly declaring the parameter (`a: Int`), omit the function arrow (`->`), and just have the function body. Update the `triple` function declared in your `main` function and run the code.

```
val triple: (Int) -> Int = { it * 3 }
```

4. The output should be the same, but now your lambda is written more concisely! For more examples of lambdas, check out this [resource](#).

15

Higher-order functions

Now that you are starting to see the flexibility of how you can manipulate functions in Kotlin, let's talk about another really powerful idea, a *higher-order function*. This just means passing a function (in this case a lambda) to another function, or returning a function from another function.

It turns out that `map`, `filter`, and `forEach` functions are all examples of higher-order functions because they all took a function as a parameter. (In the lambda passed to this `filter` higher-order function, it's okay to omit the single parameter and arrow symbol, and also use the `it` parameter.)

```
peopleAges.filter { it.key.length < 4 }
```

Here's an example of a new higher-order function: `sortedWith()`.

If you want to sort a list of strings, you can use the built-in `sorted()` method for collections. However, if you wanted to sort the list by the length of the strings, you need to write some code to get the length of two strings and compare them. Kotlin lets you do this by passing a lambda to the `sortedWith()` method.

Note: To compare two objects for sorting, the convention is to return a value less than 0 if the first object is less than the second, 0 if they are equal, and a value greater than 0 if the first object is greater than the second.

1. In the playground, create a list of names and print it sorted by name with this code:

```
fun main() {  
    val peopleNames = listOf("Fred", "Ann", "Barbara", "Joe")  
    println(peopleNames.sorted())  
}
```

2. Now print the list sorted by the length of the names by passing a lambda to the `sortedWith()` function. The lambda should take in two parameters of the same type and return an `Int`. Add this line of code after the `println()` statement in the `main()` function.

```
println(peopleNames.sortedWith { str1: String, str2: String -> str1.length -  
str2.length })
```

3. Run your program and look at the results.

```
[Ann, Barbara, Fred, Joe]  
[Ann, Joe, Fred, Barbara]
```

The lambda passed to `sortedWith()` has two parameters, `str1` which is a `String`, and `str2` which is a `String`. Then you see the function arrow, followed by the function body.

```
{ str1: String, str2: String -> str1.length - str2.length }
```

Remember that the last expression in the lambda is the return value. In this case, it returns the difference between the length of the first string and the length of the second string, which is an `Int`. That matches what is needed for sorting: if `str1` is shorter than `str2`, it will return a value less than 0. If `str1` and `str2` are the same length, it will return 0. If `str1` is longer than `str2`, it

will return a value greater than 0. By doing a series of comparison between two `Strings` at a time, the `sortedWith()` function outputs a list where the names will be in order of increasing length.

OnClickListener and OnKeyListener in Android

Tying this back to what you have learned in Android so far, you have used lambdas in earlier codelabs, such as when you set a click listener for the button in the Tip Calculator app:

```
calculateButton.setOnClickListener{ calculateTip() }
```

Using a lambda to set the click listener is convenient shorthand. The long form way of writing the above code is shown below, and compared against the shortened version. You don't have to understand all the details of the long form version, but notice some patterns between the two versions.

```
LONG FORM    calculateButton.setOnClickListener(object: View.OnClickListener {  
                override fun onClick(view: View?) {  
                    calculateTip()  
                }  
            })
```

```
SHORT FORM    calculateButton.setOnClickListener { view -> calculateTip() }
```

Observe how the lambda has the same function type as the `onClick()` method in `OnClickListener` (takes in one `View` argument and returns `Unit`, which means no return value).

The shortened version of the code is possible because of something called SAM (Single-Abstract-Method) conversion in Kotlin. Kotlin converts the lambda into an `OnClickListener` object which implements the single abstract method `onClick()`. You just need to make sure the lambda function type matches the function type of the abstract function.

Since the `view` parameter is never used in the lambda, the parameter can be omitted. Then we just have the function body in the lambda.

```
calculateButton.setOnClickListener { calculateTip() }
```

These concepts are challenging, so be patient with yourself as it'll take time and experience for these concepts to sink in. Let's look at another example. Recall when you set a key listener on the "Cost of service" text field in the tip calculator, so the onscreen keyboard could be hidden when the Enter key is pressed.

```
costOfServiceEditText.setOnKeyListener { view, keyCode, event ->  
    handleKeyEvent(view, keyCode) }
```

When you look up [OnKeyListener](#), the abstract method has the following parameters `onKey(View v, int keyCode, KeyEvent event)` and returns a `Boolean`. Because of SAM conversions in Kotlin, you can pass in a lambda to `setOnKeyListener()`. Just be sure the lambda has the function type `(View, Int, KeyEvent) -> Boolean`.

Here's a diagram of the lambda expression used above. The parameters are `view`, `keyCode`, and `event`. The function body consists of `handleKeyEvent(view, keyCode)` which uses the parameters passed in and returns a `Boolean`.

```
{ view, keyCode, event -> handleKeyEvent(view, keyCode) }
```

Note: If you don't use a lambda parameter in the function body, you can name it `_` to make your code more readable and less cluttered. This code has the same behavior.

```
costOfServiceEditText.setOnKeyListener { view, keyCode, _ ->  
    handleKeyEvent(view, keyCode) }
```

5. Make word lists

Now let's take everything you learned about collections, lambdas, and higher order functions and apply it to a realistic use case.

Suppose you wanted to create an Android app to play a word game or learn vocabulary words. The app might look something like this, with a button for each letter of the alphabet:



Clicking on the letter **A** would bring up a short list of some words that begin with the letter A, and so on.

You'll need a collection of words, but what kind of collection? If the app is going to include some words that start with each letter of the alphabet, you'll need a way to find or organize all the words that start with a given letter. To make it more challenging, you'll want to choose different words from your collection each time the user runs the app.

First, start with a list of words. For a real app you'd want a longer list of words, and include words that start with all the letters of the alphabet, but a short list is enough to work with for now.

1. Replace the code in the Kotlin playground with this code:

```
fun main() {  
    val words = listOf("about", "acute", "awesome", "balloon", "best",  
"brief", "class", "coffee", "creative")  
}
```

2. To get a collection of the words that start with the letter B, you can use `filter` with a lambda expression. Add these lines:

```
val filteredWords = words.filter { it.startsWith("b", ignoreCase = true) }  
println(filteredWords)
```

The [startsWith\(\)](#) function returns true if a string starts with the specified string. You can also tell it to ignore case, so "b" will match "b" or "B".

3. Run your program and look at the result:

```
[balloon, best, brief]
```

4. Remember that you want the words randomized for your app. With Kotlin collections, you can use the [shuffled\(\)](#) function to make a copy of a collection with the items randomly shuffled. Change the filtered words to be shuffled, too:

```
val filteredWords = words.filter { it.startsWith("b", ignoreCase = true) }  
    .shuffled()
```

5. Run your program and look at the new results:

```
[brief, balloon, best]
```

Because the words are randomly shuffled, you may see the words in a different order.

6. You don't want all the words (especially if your real word list is long), just a few. You can use the [take\(\)](#) function to get the first items in the collection. Make the filtered words just include the first two shuffled words:

```
val filteredWords = words.filter { it.startsWith("b", ignoreCase = true) }  
    .shuffled()  
    .take(2)
```

7. Run your program and look at the new results:

```
[brief, balloon]
```

Again because of the random shuffling, you might see different words each time you run it.

8. Finally, for the app you want the random list of words for each letter sorted. As before, you can use the [`sorted\(\)`](#) function to return a copy of the collection with the items sorted:

```
val filteredWords = words.filter { it.startsWith("b", ignoreCase = true) }  
    .shuffled()  
    .take(2)  
    .sorted()
```

9. Run your program and look at the new results:

```
[balloon, brief]
```

All the code above:

```
fun main() {  
    val words = listOf("about", "acute", "awesome", "balloon", "best",  
"brief", "class", "coffee", "creative")  
    val filteredWords = words.filter { it.startsWith("b", ignoreCase = true) }  
}  
    .shuffled()  
    .take(2)  
    .sorted()  
    println(filteredWords)  
}
```

10. Try changing the code to create a list of one random word that starts with the letter c. What do you have to change in the code above?

```
val filteredWords = words.filter { it.startsWith("c", ignoreCase = true) }  
    .shuffled()  
    .take(1)
```

In the actual app, you'll need to apply the filter for each letter of the alphabet, but now you know how to generate the word list for each letter!

Collections are powerful and flexible. There's a lot they can do, and there can be more than one way to do something. As you learn more about programming, you'll learn how to figure out which type of collection is right for the problem at hand and the best ways to process it.

Lambdas and higher-order functions make working with collections easier and more concise. These ideas are very useful, so you'll see them used again and again.

6. Summary

- A collection is a group of related items
- Collections can be mutable or immutable

- Collections can be ordered or unordered
- Collections can require unique items or allow duplicates
- Kotlin supports different kinds of collections including lists, sets, and maps
- Kotlin provides many functions for processing and transforming collections, including `forEach`, `map`, `filter`, `sorted`, and more.
- A lambda is a function without a name that can be passed as an expression immediately. An example would be `{ a: Int -> a * 3 }`.
- A higher-order function means passing a function to another function, or returning a function from another function.

7. Learn more

- [Vocabulary for Android Basics in Kotlin](#)
- [Kotlin collections](#)
- [List class](#)
- [Set class](#)
- [Map class](#)
- [Collection transformations](#)
- [Higher-Order Functions and Lambdas](#)
- [Function Types](#)
- [it: implicit name for single parameter](#)
- [Lambda Functions](#)
- [Higher-Order Functions](#)